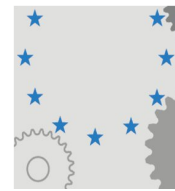


UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
Carrera de Ingeniería de Software · Aplicaciones Distribuidas

SISTEMAS DISTRIBUIDOS

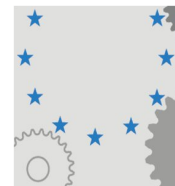
Historia, fundamentos, tipos, tecnologías y comparación con las arquitecturas
por capas y MVC

Autor: Cristian Cola
Sangolquí, septiembre de 2026



Contenido

1. Introducción y propósito.....	4
2. ¿Qué es un sistema distribuido?.....	4
2.1 Características principales.....	4
2.2 Transparencia.....	5
2.3 Las falacias de la computación distribuida.....	5
3. Historia de los sistemas distribuidos.....	6
3.1 Mainframes y tiempo compartido (1950–1970).....	6
3.2 Redes y los primeros protocolos (1969–1983).....	6
3.3 Cliente-servidor y llamadas remotas (1980–1995).....	6
3.4 La Web y la arquitectura multinivel (1989–2000).....	7
3.5 Peer-to-peer y cómputo voluntario (1999–2005).....	7
3.6 Big data y nube (2003–2012).....	7
3.7 Contenedores, microservicios y nube nativa (2013–hoy).....	7
4. Conceptos clave que todo diseño distribuido enfrenta.....	8
4.1 Escalabilidad.....	8
4.2 Tolerancia a fallos y replicación.....	8
4.3 Consistencia.....	8
4.4 El teorema CAP.....	9
4.5 Tiempo, orden y consenso.....	9
5. Ventajas y desventajas.....	9
6. Tipos de sistemas distribuidos.....	10
6.1 Según su propósito.....	10
6.2 Según su arquitectura.....	11
7. Cómo funcionan algunos sistemas reales.....	12
7.1 Una búsqueda en la Web.....	12
7.2 Netflix: microservicios y resiliencia.....	12
7.3 Chat y notificaciones en tiempo real.....	13
7.4 Clúster de alto rendimiento.....	13
7.5 Subastas en línea.....	13
8. Tecnologías y componentes.....	13
8.1 Comunicación síncrona.....	14
8.2 Tiempo real.....	14
8.3 Mensajería asíncrona y colas.....	14
8.4 Datos: replicación y particionamiento.....	14
8.5 Contenedores y orquestación.....	14
8.6 Observabilidad.....	14



9. Capas, MVC y arquitectura distribuida: comparación	15
9.1 Programación por capas	15
9.2 Patrón MVC	15
9.3 Otras arquitecturas de organización	16
9.4 Programación distribuida	16
9.5 Tabla comparativa	16
9.6 La idea clave: se combinan	17
10. Hacia dónde vamos: el sistema de subastas en tiempo real	17
10.1 Componentes y responsabilidades	18
10.2 Recorrido de una puja	18
11. Tendencias actuales	19
12. Glosario	19
Referencias	19

1. Introducción y propósito

Casi todo el software que usamos a diario es distribuido: la banca móvil, las redes sociales, las plataformas de streaming, el sistema académico de la universidad o una tienda en línea. Detrás de cada clic participan varios computadores que se comunican por red y cooperan para dar una respuesta.

Hasta ahora, en asignaturas anteriores, se ha construido software que se ejecuta en un solo lugar: una aplicación organizada por capas o con el patrón Modelo–Vista–Controlador (MVC), que corre en un servidor y usa una base de datos. Este documento explica qué cambia cuando la aplicación se reparte entre varias máquinas, por qué se hace, qué problemas nuevos aparecen y con qué componentes se resuelven.

Hacia dónde vamos. Durante el primer parcial se construirá un sistema de **subastas en tiempo real** con Laravel. Una subasta es un buen caso de estudio: muchos usuarios pujan al mismo tiempo, todos deben ver la puja más alta al instante, dos pujas simultáneas no pueden ganar a la vez y la subasta debe cerrarse sola cuando vence el plazo. Resolver eso obliga a usar los conceptos de este documento: comunicación entre procesos, eventos, colas, WebSockets, concurrencia y escalabilidad.

Al terminar la lectura, el estudiante debería poder:

- Definir qué es un sistema distribuido y reconocer sus características.
- Explicar su evolución histórica y las tecnologías que marcaron cada etapa.
- Identificar ventajas, desventajas y tipos de sistemas distribuidos con ejemplos reales.
- Diferenciar la arquitectura por capas, el patrón MVC y la arquitectura distribuida, y entender cómo se combinan.
- Reconocer los componentes que se usarán en el proyecto de subastas y el papel de cada uno.

2. ¿Qué es un sistema distribuido?

Van Steen y Tanenbaum (2023) lo definen como **una colección de elementos de cómputo autónomos que ante sus usuarios aparece como un único sistema coherente**. La definición tiene dos partes:

- **Elementos autónomos:** cada nodo (un servidor, un contenedor, un teléfono, un sensor) tiene su propio procesador, memoria y reloj. Se coordinan únicamente intercambiando mensajes por la red; no comparten memoria.
- **Sistema único y coherente:** el usuario no debería notar que hay muchas máquinas. Cuando se abre una aplicación de mensajería no se piensa en cuántos servidores atienden la conversación.

2.1 Características principales

Característica	Qué significa	Ejemplo
Concurrencia	Muchos procesos se ejecutan al mismo tiempo en nodos distintos y acceden a recursos compartidos.	Cien usuarios pujan por el mismo artículo.
Sin reloj global	Cada máquina tiene su propio reloj; no existe una hora exacta común.	Dos pujas con la misma hora: ¿cuál llegó primero?
Fallos independientes	Un nodo puede caerse mientras los demás siguen	Cae el servidor de notificaciones,

Característica	Qué significa	Ejemplo
	funcionando (fallo parcial).	pero se puede seguir pujando.
Comunicación por mensajes	Los nodos solo se coordinan enviando mensajes por la red.	Petición HTTP, mensaje en una cola, evento por WebSocket.
Heterogeneidad	Hardware, sistemas operativos y lenguajes diferentes conviven.	Servidor Linux en PHP, cliente Android, app iOS.

2.2 Transparencia

Un objetivo de diseño es **ocultar la distribución** al usuario y, en lo posible, al programador. Esa propiedad se llama transparencia y tiene varias formas (van Steen y Tanenbaum, 2023):

Tipo	Oculto...
Acceso	Cómo se representan y accede a los datos (formatos, sistemas operativos).
Ubicación	En qué máquina física está el recurso. Una URL no revela el servidor que responde.
Reubicación / migración	Que un recurso o proceso se movió a otro lugar mientras se usaba.
Replicación	Que existen varias copias de un mismo dato o servicio.
Concurrencia	Que otros usuarios comparten el mismo recurso al mismo tiempo.
Fallos	Que un componente falló y el sistema se recuperó.

La transparencia total es imposible y a veces ni siquiera es deseable: la red introduce retardos que no se pueden esconder. Un buen diseño decide qué ocultar y qué mostrar (por ejemplo, un mensaje de “reconectando...”).

2.3 Las falacias de la computación distribuida

A mediados de los años noventa, Peter Deutsch y otros ingenieros de Sun Microsystems resumieron en una lista los supuestos falsos que cometen quienes programan su primer sistema distribuido. Siguen vigentes y conviene memorizarlos (Vitillo, 2022):

1. La red es confiable.
2. La latencia es cero.
3. El ancho de banda es infinito.
4. La red es segura.
5. La topología no cambia.
6. Hay un solo administrador.
7. El costo de transporte es cero.
8. La red es homogénea.

Cada falacia se traduce en un error concreto de programación: no manejar tiempos de espera (*timeouts*), enviar datos de más, confiar en mensajes sin autenticar o suponer que un servidor siempre tiene la misma dirección IP.

Dato curioso · La definición más citada es una broma

Leslie Lamport, Premio Turing 2013 y uno de los padres de esta área, escribió en 1987 en un correo interno: “Un sistema distribuido es aquel en el que la falla de un computador cuya existencia ni siquiera conocías puede dejar inutilizable tu propio computador”. La frase sigue apareciendo en libros y cursos porque describe muy bien el fallo parcial.

3. Historia de los sistemas distribuidos

La historia se entiende mejor como una secuencia de problemas: cada etapa resolvió una limitación de la anterior y creó nuevas necesidades.

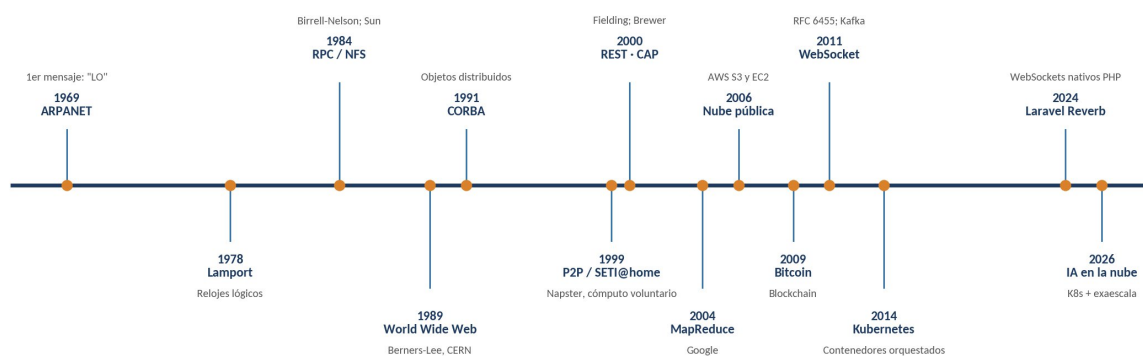


Figura 1. Hitos en la evolución de los sistemas distribuidos. Elaboración propia.

3.1 Mainframes y tiempo compartido (1950–1970)

Los primeros computadores eran grandes, caros y centralizados. Muchos usuarios compartían un único *mainframe* mediante terminales “tontas” que solo mostraban texto. Todo el procesamiento ocurría en un solo lugar: era un sistema centralizado, no distribuido.

3.2 Redes y los primeros protocolos (1969–1983)

En 1969 se conectaron los primeros nodos de **ARPANET**, financiada por el Departamento de Defensa de EE. UU. En 1973 Xerox PARC creó **Ethernet**, que permitió redes locales baratas, y en 1983 ARPANET adoptó **TCP/IP**, base de la Internet actual. Aparecen los problemas teóricos fundamentales: en 1978 Lamport publica cómo ordenar eventos sin un reloj común (relojes lógicos) y en 1982 Lamport, Shostak y Pease plantean el problema de los generales bizantinos, que estudia cómo llegar a acuerdos cuando algunos nodos fallan o mienten.

Dato curioso · El primer mensaje de Internet fue “LO”

El 29 de octubre de 1969, un estudiante de UCLA intentó escribir “LOGIN” en un computador del Instituto de Investigación de Stanford. El sistema se cayó después de la segunda letra. El primer mensaje transmitido por ARPANET fue, entonces, “LO”. El primer sistema distribuido de la historia sufrió un fallo parcial en su primer minuto.

3.3 Cliente-servidor y llamadas remotas (1980–1995)

Los computadores personales y las redes locales dieron origen al modelo **cliente-servidor**: el cliente pide, el servidor responde. En 1984 Birrell y Nelson describen la **llamada a procedimiento remoto (RPC)**, que permite invocar una función en otra máquina como si fuera local, y Sun lanza **NFS**, un

sistema de archivos en red. A inicios de los noventa aparecen los objetos distribuidos (**CORBA** en 1991, luego Java RMI y DCOM), que llevaron la programación orientada a objetos a la red.

Dato curioso · “La red es el computador”

Este fue el lema de Sun Microsystems desde 1984, acuñado por John Gage. Era una idea radical para su época: el valor no estaba en una máquina aislada sino en la red que las conectaba. Hoy es una descripción literal de la nube.

3.4 La Web y la arquitectura multinivel (1989–2000)

En 1989 Tim Berners-Lee propuso en el CERN un sistema de documentos enlazados: la **World Wide Web**. La Web convirtió al navegador en un cliente universal y popularizó la arquitectura de **tres niveles** (navegador, servidor de aplicaciones y servidor de base de datos). En 2000 Roy Fielding describió en su tesis doctoral el estilo **REST**, que hoy domina la comunicación entre servicios, y Eric Brewer presentó el **teorema CAP** (sección 4.4).

Dato curioso · “Vago, pero emocionante”

Mike Sendall, jefe de Tim Berners-Lee en el CERN, escribió a mano sobre la portada de la propuesta de 1989: “Vague but exciting...”. Con ese comentario aprobó el proyecto que terminaría siendo la Web.

3.5 Peer-to-peer y cómputo voluntario (1999–2005)

Napster (1999) y luego BitTorrent (2001) demostraron que millones de computadores personales podían compartir recursos sin un servidor central. SETI@home (1999) usó el tiempo libre de los PC de voluntarios para analizar señales de radio en busca de vida extraterrestre. Ese mismo año se consolidó el concepto de **grid computing**, que unía recursos de distintas instituciones para cómputo científico.

3.6 Big data y nube (2003–2012)

Google publicó cómo funcionaba su sistema de archivos distribuido (GFS, 2003) y su modelo de procesamiento **MapReduce** (2004); a partir de ellos nació Hadoop (2006). En 2006 Amazon lanzó S3 y EC2, el inicio de la **nube pública**: cualquiera podía alquilar servidores por horas. En 2009 comenzó a funcionar **Bitcoin**, un libro contable distribuido sin autoridad central. En 2011 se estandarizó el protocolo **WebSocket** (RFC 6455), que permite comunicación bidireccional permanente entre navegador y servidor, y LinkedIn liberó **Kafka**, una plataforma de mensajería para flujos de eventos.

Dato curioso · Un elefante de juguete

Hadoop se llama así por el elefante de peluche amarillo del hijo de Doug Cutting, uno de sus creadores. Por eso el logo del proyecto es un elefante.

3.7 Contenedores, microservicios y nube nativa (2013–hoy)

Docker (2013) empaquetó aplicaciones en **contenedores** portables y Google liberó **Kubernetes** (2014) para orquestarlos, a partir de su experiencia interna con el sistema Borg. Empresas como Netflix y Amazon popularizaron los **microservicios**: aplicaciones divididas en decenas o cientos de servicios pequeños. Según la encuesta anual de la CNCF publicada en enero de 2026, el 82 % de las

organizaciones que usan contenedores ya ejecuta Kubernetes en producción (Cloud Native Computing Foundation [CNCF], 2026).

En el ecosistema PHP, Laravel 11 (marzo de 2024) incorporó **Reverb**, un servidor WebSocket propio escrito en PHP, lo que permite construir aplicaciones en tiempo real sin depender de servicios externos (Laravel, 2026b). Es la herramienta que usaremos en el proyecto.

En el extremo del cómputo de alto rendimiento, la lista TOP500 de junio de 2026 ubicó en primer lugar a LineShine (Shenzhen, China), con 2,198 exaflops sostenidos y casi 13,8 millones de núcleos trabajando coordinadamente (TOP500, 2026). Una supercomputadora moderna también es un sistema distribuido: miles de nodos unidos por una red de alta velocidad.

Dato curioso · ¿Por qué K8s?

Kubernetes viene del griego y significa “timonel”. Se abrevia K8s porque entre la K y la s hay ocho letras. El timón de su logo tiene siete radios en homenaje a su nombre interno en Google: “Project Seven of Nine”, un personaje de Star Trek.

4. Conceptos clave que todo diseño distribuido enfrenta

4.1 Escalabilidad

Es la capacidad de atender más carga agregando recursos. Hay dos estrategias:

- **Escalamiento vertical** (*scale up*): poner una máquina más potente (más CPU, más RAM). Es simple, pero tiene un techo físico y económico.
- **Escalamiento horizontal** (*scale out*): agregar más máquinas y repartir el trabajo. No tiene un techo práctico, pero exige que la aplicación esté diseñada para funcionar en varias instancias a la vez.

Van Steen y Tanenbaum (2023) distinguen además tres dimensiones: escalar en **tamaño** (más usuarios y recursos), en **geografía** (usuarios lejanos, con más latencia) y en **administración** (varias organizaciones con políticas distintas).

4.2 Tolerancia a fallos y replicación

Si un nodo cae, otro debe tomar su lugar. La técnica base es la **replicación**: mantener copias del mismo servicio o dato en varios nodos. Replicar mejora la disponibilidad y el rendimiento, pero introduce un problema nuevo: mantener las copias sincronizadas.

4.3 Consistencia

Cuando hay varias copias de un dato, ¿todas muestran siempre el mismo valor? La **consistencia fuerte** garantiza que cualquier lectura devuelve la última escritura, a costa de coordinación y latencia. La **consistencia eventual** acepta que durante un breve lapso las copias difieran, pero garantiza que terminarán igualándose. En una subasta, la puja más alta necesita consistencia fuerte; el contador de “personas mirando este artículo” puede ser eventual (Kleppmann y Riccomini, 2026).

4.4 El teorema CAP

Formulado por Eric Brewer en 2000 y demostrado por Gilbert y Lynch en 2002, dice que cuando ocurre una **partición de red** (P), es decir, cuando parte de los nodos no puede comunicarse con el resto, el sistema debe elegir entre **consistencia** (C: responder solo con datos correctos, aunque rechace peticiones) y **disponibilidad** (A: responder siempre, aunque con datos posiblemente desactualizados). Como las particiones son inevitables en una red real, la decisión práctica es qué hacer cuando ocurren. Kleppmann y Riccomini (2026) advierten que CAP es un punto de partida útil pero simplificado; el diseño real exige analizar cada operación.

4.5 Tiempo, orden y consenso

Como no existe un reloj global, ordenar eventos es difícil. Los **relojes lógicos** de Lamport asignan números a los eventos para respetar la relación “ocurrió antes que”. Cuando varios nodos deben ponerse de acuerdo (por ejemplo, cuál es el líder o cuál fue la puja ganadora), se usan **algoritmos de consenso** como Paxos o Raft, presentes en herramientas como etcd, que es la base de datos interna de Kubernetes (Joshi, 2023).

Dato curioso · Un algoritmo disfrazado de arqueología

Lamport presentó Paxos como la historia de un parlamento en una isla griega ficticia cuyos legisladores entraban y salían del recinto. Los revisores no entendieron la broma y el artículo, escrito en 1989, recién se publicó en 1998. En 2014, Ongaro y Ousterhout diseñaron Raft con un objetivo declarado en el título de su artículo: que fuera un algoritmo de consenso “comprensible”.

5. Ventajas y desventajas

Ventajas	Desventajas
Escalabilidad: se agregan nodos a medida que crece la demanda.	Complejidad: más piezas, más configuraciones y más formas de fallar.
Disponibilidad: si un nodo cae, otro responde; no hay un único punto de falla.	Fallos parciales: una parte funciona y otra no; el diagnóstico es difícil.
Rendimiento: el trabajo se reparte y se ejecuta en paralelo.	Latencia de red: una llamada remota es miles de veces más lenta que una local.
Compartir recursos: datos, impresoras, cómputo o almacenamiento entre muchos usuarios.	Consistencia: mantener copias sincronizadas tiene un costo.
Cercanía geográfica: servidores cerca del usuario (CDN, edge).	Seguridad: más puntos de entrada y datos viajando por la red.
Independencia tecnológica y de equipos: cada servicio puede usar su tecnología y evolucionar por separado.	Pruebas y depuración: reproducir un error que depende del orden de los mensajes es complicado.
Costo incremental: se paga por lo que se usa, especialmente en la nube.	Operación: requiere monitoreo, despliegue automatizado y personal capacitado.

La conclusión práctica es que **distribuir no es gratis**. Richards y Ford (2025) insisten en que toda decisión de arquitectura es un intercambio (*trade-off*): se gana escalabilidad y disponibilidad a cambio de complejidad. Un sistema pequeño con pocos usuarios suele estar mejor servido por una aplicación monolítica bien organizada.

Dato curioso · Números que todo programador debería conocer

Jeff Dean, de Google, popularizó una tabla de latencias aproximadas: leer de la memoria RAM toma unos 100 nanosegundos; ir y volver dentro del mismo centro de datos, cerca de medio milisegundo; y una petición de California a Europa y de regreso, unos 150 milisegundos. Es decir, una llamada intercontinental es más de un millón de veces más lenta que leer de memoria. Por eso “la latencia es cero” es una falacia tan cara.

6. Tipos de sistemas distribuidos

Los sistemas distribuidos se pueden clasificar de dos maneras complementarias: según **para qué se usan** y según **cómo se organizan** sus componentes.

6.1 Según su propósito

Van Steen y Tanenbaum (2023) proponen tres grandes familias:

a) Sistemas de cómputo distribuido

- **Clúster:** computadores similares, en la misma sala, unidos por una red rápida y administrados como un solo equipo. Un gestor de trabajos (como Slurm) reparte las tareas entre los nodos. Ejemplo: las supercomputadoras del TOP500 y los clústeres de laboratorios universitarios.
- **Grid:** recursos de varias instituciones, heterogéneos y en distintos lugares, que se federan para un objetivo común. Ejemplo: la red mundial de cómputo del CERN para procesar datos del Gran Colisionador de Hadrones.
- **Nube (cloud):** recursos virtualizados que se alquilan bajo demanda como infraestructura (IaaS), plataforma (PaaS) o software (SaaS). Ejemplo: AWS, Microsoft Azure, Google Cloud.

b) Sistemas de información distribuidos

- **Procesamiento de transacciones:** varias operaciones en distintas bases de datos deben ejecutarse todas o ninguna (atomicidad). Ejemplo: una transferencia entre dos bancos.
- **Integración de aplicaciones empresariales:** sistemas independientes (contabilidad, nómina, inventario) intercambian información mediante APIs o middleware de mensajería. Ejemplo: un ERP que se comunica con la pasarela de pagos y con el sistema de facturación electrónica.

c) Sistemas ubicuos o pervasivos

- **Móviles:** dispositivos que cambian de ubicación y de red. Ejemplo: aplicaciones de transporte que rastrean al conductor.
- **Redes de sensores e IoT:** muchos dispositivos pequeños, con poca energía, que miden y reportan datos. Ejemplo: estaciones meteorológicas o agricultura de precisión.
- **Edge y fog computing:** el procesamiento se acerca al lugar donde se generan los datos para reducir latencia. Ejemplo: cámaras que analizan video localmente y solo envían alertas a la nube.

Dato curioso · Slurm y Futurama

Slurm, el gestor de trabajos de muchas supercomputadoras, se presentó originalmente como Simple Linux Utility for Resource Management. El nombre también es un guiño a Slurm, la bebida ficticia de la serie Futurama.

6.2 Según su arquitectura

Cliente-servidor

Cómo funciona: el cliente envía una petición y el servidor procesa y responde. El servidor es centralizado y conocido. **Ejemplo:** un navegador que pide una página web; una app móvil que consulta el saldo bancario.

Multinivel (n-tier)

Cómo funciona: el servidor se divide en niveles físicos: servidor web, servidor de aplicaciones y servidor de base de datos, cada uno en su propia máquina. **Ejemplo:** la mayoría de sistemas académicos y empresariales web.

Peer-to-peer (P2P)

Cómo funciona: todos los nodos son iguales; cada uno es cliente y servidor a la vez. No hay un centro que pueda caer. **Ejemplo:** BitTorrent divide un archivo en fragmentos; cada usuario descarga fragmentos de otros y, al mismo tiempo, comparte los que ya tiene. Mientras más gente descarga, más rápido se distribuye.

Arquitectura orientada a servicios (SOA)

Cómo funciona: la funcionalidad se expone como servicios reutilizables, normalmente conectados por un bus de servicios empresarial (ESB) y contratos formales (SOAP, WSDL). **Ejemplo:** grandes corporaciones y entidades públicas que integran sistemas heredados.

Microservicios

Cómo funciona: la aplicación se divide en servicios pequeños, cada uno responsable de una capacidad del negocio, con su propia base de datos, desplegado de forma independiente y comunicado por HTTP/REST, gRPC o mensajería. **Ejemplo:** Netflix tiene servicios separados para catálogo, recomendaciones, reproducción, facturación y perfiles; si falla el de recomendaciones, la película se sigue reproduciendo.

Orientada a eventos

Cómo funciona: los componentes no se llaman directamente; publican **eventos** ("se registró una puja") y otros componentes suscritos reaccionan. El emisor no sabe quién escucha. **Ejemplo:** al confirmar una compra en línea se publica un evento y, de forma independiente, se descuenta inventario, se envía un correo y se genera la factura.

Serverless (funciones como servicio)

Cómo funciona: el programador sube funciones pequeñas y la nube las ejecuta solo cuando ocurre un evento, escalando de cero a miles de instancias automáticamente. Se paga por ejecución. **Ejemplo:** una función que redimensiona cada imagen que se sube a un almacenamiento.

Blockchain

Cómo funciona: una red P2P mantiene un registro de transacciones replicado en todos los nodos; cada bloque se enlaza criptográficamente al anterior y un mecanismo de consenso decide qué bloque se agrega. **Ejemplo:** Bitcoin, que funciona sin banco central.

Arquitectura	Centro de control	Comunicación típica	Ejemplo
Cliente-servidor	Servidor	Petición/respuesta (HTTP)	Sitio web
Multinivel	Servidores por nivel	HTTP + conexión a BD	Sistema académico
P2P	Ninguno	Mensajes entre pares	BitTorrent
SOA	Bus de servicios (ESB)	SOAP / XML	Integración gubernamental
Microservicios	Distribuido por servicio	REST, gRPC, colas	Netflix, Uber
Orientada a eventos	Broker de mensajes	Publicar/suscribir	Comercio electrónico
Serverless	Proveedor de nube	Eventos y HTTP	Procesar imágenes
Blockchain	Consenso de la red	P2P + criptografía	Bitcoin

Dato curioso · La ley de Conway

En 1968 Melvin Conway observó que las organizaciones diseñan sistemas que copian su propia estructura de comunicación. Si cuatro equipos construyen un compilador, obtendrán un compilador de cuatro fases. Las empresas que adoptan microservicios suelen reorganizarse en equipos pequeños, uno por servicio, justamente por esta razón.

7. Cómo funcionan algunos sistemas reales

7.1 Una búsqueda en la Web

Al escribir una dirección y presionar Enter ocurre una cadena de interacciones distribuidas:

1. El navegador pregunta al **DNS**, una base de datos distribuida y jerárquica, qué dirección IP corresponde al nombre.
2. La petición llega a un **balanceador de carga**, que elige uno de muchos servidores disponibles.
3. El servidor consulta **cachés** y **servicios internos** (índice, anuncios, sugerencias), cada uno replicado en muchas máquinas.
4. Las imágenes y archivos estáticos se entregan desde una **CDN**, una red de servidores ubicados cerca del usuario.

El usuario percibe una sola respuesta, en milisegundos: eso es transparencia.

7.2 Netflix: microservicios y resiliencia

En 2008 una corrupción de su base de datos detuvo durante tres días el envío de DVD de Netflix. La empresa decidió migrar a la nube de Amazon y rediseñar su sistema como microservicios, un proceso que completó en 2016. Para probar que el sistema resiste fallos, crearon herramientas que **apagan servidores al azar en producción**.

Dato curioso · Un mono en el centro de datos

La herramienta de Netflix se llama Chaos Monkey: imagina un mono suelto en un centro de datos desconectando cables. Se liberó como código abierto en 2012 y dio origen a una disciplina completa, la ingeniería del caos. La idea: si los fallos van a ocurrir, mejor provocarlos en horario laboral, cuando los ingenieros están despiertos.

7.3 Chat y notificaciones en tiempo real

En una aplicación de chat, el cliente abre una **conexión WebSocket** que se mantiene abierta. Cuando alguien envía un mensaje, el servidor lo guarda, lo publica en un canal y lo **empuja** (*push*) a todos los clientes suscritos, sin que ellos tengan que preguntar. Es exactamente el mecanismo que usará el sistema de subastas para avisar a todos los postores de una nueva puja.

7.4 Clúster de alto rendimiento

En un clúster HPC, el usuario envía un trabajo a un **gestor de colas** (por ejemplo, Slurm). El gestor decide en qué nodos se ejecuta según los recursos libres; los procesos se comunican entre nodos mediante bibliotecas como MPI y escriben en un sistema de archivos paralelo compartido. Aquí el objetivo no es atender muchos usuarios, sino resolver un único problema enorme dividiéndolo en partes.

7.5 Subastas en línea

Una plataforma de subastas combina varios de los mecanismos anteriores: peticiones HTTP para pujar, una base de datos transaccional para garantizar que solo una puja gane, eventos para notificar, WebSockets para actualizar las pantallas y tareas programadas para cerrar subastas. Es el proyecto de este parcial y se detalla en la sección 10.

8. Tecnologías y componentes

Un sistema distribuido se arma con piezas que cumplen funciones específicas. La tabla resume las más usadas; luego se explica cada grupo.

Función	Tecnologías	Para qué sirve
Comunicación síncrona	HTTP/REST, gRPC, GraphQL	Pedir algo y esperar la respuesta.
Tiempo real	WebSocket, Server-Sent Events	El servidor empuja datos al cliente al instante.
Mensajería asíncrona	RabbitMQ, Apache Kafka, Redis, Amazon SQS	Enviar trabajos o eventos sin esperar respuesta.
Datos	MySQL, PostgreSQL, MongoDB, Cassandra	Almacenar y replicar información.
Caché	Redis, Memcached	Guardar en memoria datos de acceso frecuente.
Entrada y balanceo	Nginx, HAProxy, API Gateway	Repartir peticiones y centralizar la entrada.
Contenedores y orquestación	Docker, Kubernetes	Empaquetar, desplegar y escalar servicios.
Coordinación	etcd, Apache ZooKeeper, Consul	Configuración compartida, elección de líder, descubrimiento de servicios.
Observabilidad	OpenTelemetry, Prometheus, Grafana	Métricas, registros y trazas entre servicios.
Seguridad	TLS, OAuth 2.0, JWT	Cifrar, autenticar y autorizar.

8.1 Comunicación síncrona

REST usa los verbos de HTTP (GET, POST, PUT, DELETE) sobre recursos identificados por URL; es simple y universal. **gRPC**, liberado por Google en 2015, usa mensajes binarios y contratos tipados, lo que lo hace más rápido para comunicación entre servicios internos. **GraphQL**, liberado por Facebook ese mismo año, permite al cliente pedir exactamente los campos que necesita. La limitación común: el cliente queda esperando y, si el servidor no responde, hay que manejar el tiempo de espera.

8.2 Tiempo real

En HTTP clásico el servidor solo responde cuando le preguntan. Para saber si hay una nueva puja, el cliente tendría que consultar cada segundo (*polling*), lo que desperdicia recursos. El protocolo **WebSocket** mantiene una conexión abierta y bidireccional: el servidor envía datos cuando ocurren. En Laravel, **Reverb** actúa como servidor WebSocket y **Laravel Echo** es la biblioteca JavaScript con la que el navegador se suscribe a canales (Laravel, 2026a, 2026b).

8.3 Mensajería asíncrona y colas

Una **cola** desacopla a quien produce un trabajo de quien lo ejecuta. La aplicación deja un mensaje (“enviar correo al ganador”) y responde de inmediato al usuario; un proceso trabajador (*worker*) lo toma y lo ejecuta después. Si el trabajador falla, el mensaje sigue en la cola y se reintenta. **RabbitMQ** implementa colas clásicas; **Kafka** guarda un registro ordenado y persistente de eventos que muchos consumidores pueden leer. Laravel ofrece colas con varios controladores (base de datos, Redis, Amazon SQS) mediante el comando `php artisan queue:work` (Laravel, 2026c).

Dato curioso · Kafka, por Franz Kafka

Jay Kreps, uno de sus creadores en LinkedIn, contó que eligió el nombre porque era un sistema optimizado para escribir, y Franz Kafka era un escritor que le gustaba. La comparación con los procesos kafkianos llegó después, por cuenta de los usuarios.

8.4 Datos: replicación y particionamiento

Para escalar los datos se usan dos técnicas combinadas (Kleppmann y Riccomini, 2026). La **replicación** copia los mismos datos en varios nodos: un nodo principal recibe escrituras y las réplicas atienden lecturas. El **particionamiento** (*sharding*) divide los datos en partes y cada nodo guarda una porción, por ejemplo, usuarios A–M en un servidor y N–Z en otro. **Redis** complementa como almacén en memoria para caché, sesiones, colas y **bloqueos atómicos**, útiles para impedir que dos procesos modifiquen la misma subasta al mismo tiempo.

8.5 Contenedores y orquestación

Un **contenedor** empaqueta la aplicación con todas sus dependencias; se ejecuta igual en la máquina del estudiante, en el laboratorio y en la nube, lo que elimina el clásico “en mi máquina sí funciona”. **Kubernetes** administra cientos de contenedores: los reinicia si fallan, crea más instancias cuando sube la carga y distribuye el tráfico entre ellas (Burns, 2025).

8.6 Observabilidad

En un monolito, un error deja una traza en un archivo. En un sistema distribuido una petición atraviesa varios servicios, y encontrar dónde falló exige **trazas distribuidas**: un identificador que viaja con la

petición por todos los servicios. **OpenTelemetry** es hoy el estándar abierto para recolectar métricas, registros y trazas; según la CNCF (2026) es el segundo proyecto con mayor actividad de su fundación.

9. Capas, MVC y arquitectura distribuida: comparación

Esta es la sección central para entender el paso que daremos en la asignatura. La confusión más frecuente es creer que la arquitectura por capas, el patrón MVC y la arquitectura distribuida son alternativas entre las que hay que elegir. **No lo son: responden a preguntas diferentes.**

- La arquitectura por capas y el MVC responden: **¿cómo organizo el código?**
- La arquitectura distribuida responde: **¿dónde y en cuántos procesos se ejecuta el sistema, y cómo se comunican?**

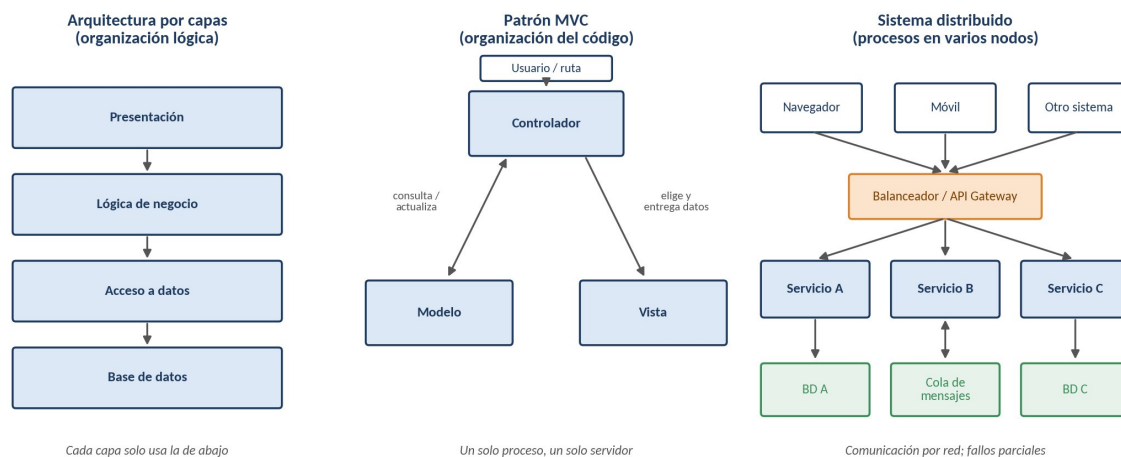


Figura 2. Capas, MVC y sistema distribuido: tres niveles de decisión distintos. Elaboración propia.

9.1 Programación por capas

Divide la aplicación en capas horizontales con responsabilidades separadas. La forma clásica tiene tres o cuatro: **presentación** (lo que ve el usuario), **lógica de negocio** (reglas: "una puja debe superar a la actual"), **acceso a datos** (consultas) y la **base de datos**. Cada capa solo se comunica con la inmediatamente inferior (Richards y Ford, 2025).

Capa no es lo mismo que nivel. En inglés se distingue *layer* (capa lógica, organización del código) de *tier* (nivel físico, dónde se ejecuta). Una aplicación de tres capas puede correr completa en una sola máquina (un nivel) o repartida en tres servidores (tres niveles). Solo en el segundo caso hay distribución. Esta distinción es la puerta de entrada a los sistemas distribuidos.

9.2 Patrón MVC

Separa la aplicación en tres roles: el **Modelo** representa los datos y las reglas del dominio; la **Vista** presenta la información; el **Controlador** recibe la acción del usuario, coordina al modelo y elige la vista. Su objetivo es que la interfaz y la lógica puedan cambiar por separado.

Laravel es un framework MVC: las **rutas** dirigen la petición a un **controlador**, los **modelos Eloquent** representan las tablas y las **vistas Blade** generan el HTML. Una aplicación Laravel típica es, entonces, un monolito MVC: un solo proceso, en un solo servidor, con una base de datos.

Dato curioso · MVC tiene casi 50 años

Trygve Reenskaug lo propuso entre 1978 y 1979 mientras trabajaba con Smalltalk en Xerox PARC, el mismo lugar donde nacieron Ethernet y la interfaz gráfica con ventanas. Su primer nombre fue Thing-Model-View-Editor. Fue pensado para interfaces de escritorio, mucho antes de que existiera la Web.

9.3 Otras arquitecturas de organización

- **Monolito modular:** una sola aplicación desplegable, pero dividida internamente en módulos por dominio (subastas, usuarios, pagos) con fronteras claras. Es un paso intermedio recomendado antes de pensar en microservicios (Richards y Ford, 2025).
- **Arquitectura hexagonal (puertos y adaptadores) y arquitectura limpia:** colocan la lógica de negocio en el centro y aíslan la base de datos, la web y los servicios externos mediante interfaces. Facilitan que una parte del sistema se convierta después en un servicio remoto.
- **Cliente-servidor, SOA y microservicios:** ya son arquitecturas de despliegue distribuido (sección 6.2).

9.4 Programación distribuida

En la programación distribuida el sistema se divide en **procesos independientes** que se ejecutan en nodos distintos y se comunican por mensajes. El programador debe asumir que cualquier llamada puede tardar, fallar o llegar duplicada, y que otros procesos modifican los mismos datos al mismo tiempo. Aparecen preocupaciones que en un monolito no existen: tiempos de espera, reintentos, idempotencia (que repetir una operación no cause un efecto doble), consistencia entre copias y monitoreo de muchos componentes (Vitillo, 2022).

9.5 Tabla comparativa

Criterio	Por capas	MVC	Distribuida
Pregunta que responde	¿Cómo separo responsabilidades?	¿Cómo separo interfaz, control y datos?	¿En qué procesos y nodos se ejecuta?
Unidad de división	Capa horizontal	Rol (modelo, vista, controlador)	Proceso o servicio
Comunicación interna	Llamadas a funciones	Llamadas a métodos	Mensajes por red (HTTP, colas, WebSocket)
Despliegue	Normalmente un solo artefacto	Un solo artefacto	Varios artefactos independientes
Escalamiento	Vertical o réplica completa	Vertical o réplica completa	Horizontal, por componente
Fallos	Todo o nada	Todo o nada	Parciales
Consistencia de datos	Una base, transacciones locales	Una base, transacciones locales	Varias fuentes; fuerte o eventual
Complejidad	Baja	Baja	Alta
Cuándo conviene	Aplicaciones de gestión, equipos pequeños	Aplicaciones web con interfaz	Alta concurrencia, tiempo real, disponibilidad
En Laravel	Controladores → servicios → repositorios	Rutas, controladores, Eloquent, Blade	Colas, eventos, Reverb, Redis, varias instancias

9.6 La idea clave: se combinan

Un sistema distribuido no reemplaza al MVC ni a las capas: **los contiene**. Cada nodo o servicio de un sistema distribuido sigue estando bien organizado internamente, con capas o con MVC. En el proyecto de subastas, la aplicación Laravel mantendrá su estructura MVC, pero será solo **una pieza** de un sistema mayor que incluye un servidor WebSocket, trabajadores de colas, Redis, la base de datos y un planificador de tareas.

La ruta natural de evolución de un sistema es:

1. **Monolito MVC** bien organizado (lo que ya conocen).
2. **Separación en niveles**: base de datos y servidor web en máquinas distintas.
3. **Procesos auxiliares**: colas, trabajadores y tareas programadas fuera de la petición HTTP.
4. **Tiempo real**: un servidor WebSocket que empuja eventos a los clientes.
5. **Escalamiento horizontal**: varias instancias de la aplicación detrás de un balanceador.
6. **Servicios independientes**, solo cuando el tamaño del sistema y del equipo lo justifican.

El primer parcial recorre los pasos 1 a 5.

10. Hacia dónde vamos: el sistema de subastas en tiempo real

La Figura 3 muestra la arquitectura objetivo. Cada caja es un proceso independiente; en desarrollo pueden correr todos en la misma computadora (en terminales distintas) y en producción pueden repartirse en varios servidores sin cambiar el código.

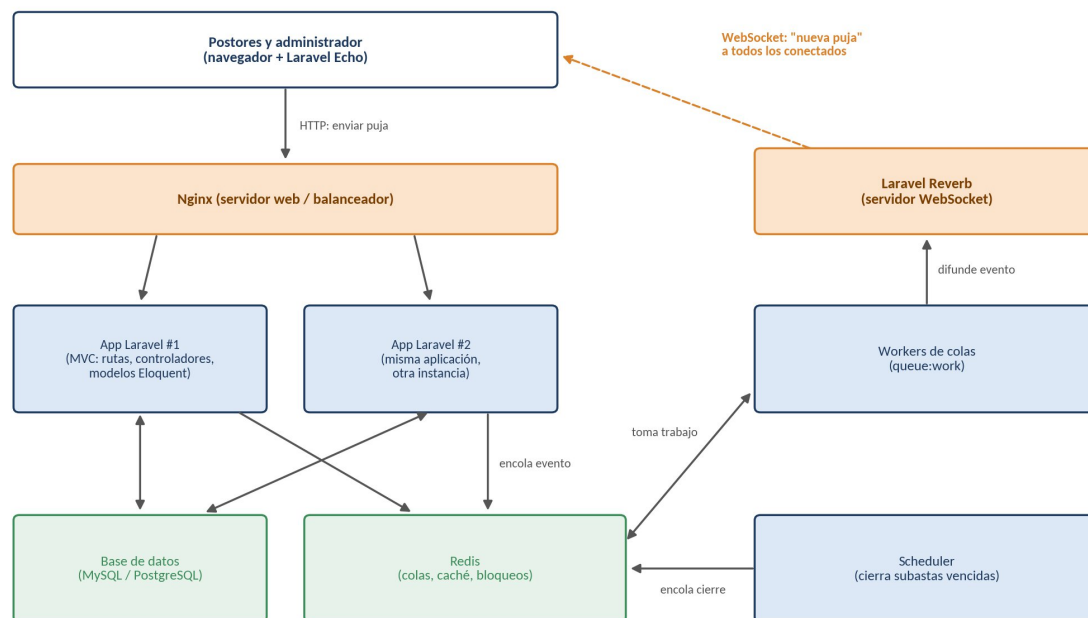


Figura 3. Arquitectura distribuida del sistema de subastas con Laravel. Elaboración propia.

10.1 Componentes y responsabilidades

Componente	Responsabilidad	Herramienta
Cliente web	Mostrar subastas, enviar pujas y recibir	Blade + JavaScript + Laravel

Componente	Responsabilidad	Herramienta
	actualizaciones sin recargar la página.	Echo
Servidor web / balanceador	Recibir peticiones HTTP y repartirlas entre instancias de la aplicación.	Nginx (o php artisan serve en desarrollo)
Aplicación	Validar la puja, aplicar reglas de negocio y guardarla.	Laravel (MVC)
Base de datos	Fuente de verdad: subastas, pujas, usuarios.	MySQL o PostgreSQL
Redis	Colas, caché y bloqueos para evitar que dos pujas simultáneas ganen.	Redis
Trabajadores de colas	Procesar eventos y tareas fuera de la petición del usuario.	php artisan queue:work
Servidor WebSocket	Mantener conexiones abiertas y difundir eventos a los clientes.	Laravel Reverb (php artisan reverb:start)
Planificador	Cerrar subastas vencidas y notificar al ganador.	Laravel Scheduler

10.2 Recorrido de una puja

1. El postor escribe un monto y presiona **Pujar**; el navegador envía una petición HTTP POST.
2. Nginx entrega la petición a una de las instancias de Laravel.
3. El **controlador** valida los datos; el **modelo** verifica, dentro de una transacción con bloqueo, que la puja supere a la actual y que la subasta siga abierta.
4. Si es válida, se guarda y se dispara un **evento** NuevaPuja que implementa la interfaz de difusión (*broadcasting*).
5. El evento pasa por la **cola**; un trabajador lo envía a **Reverb**.
6. Reverb lo **empuja** por WebSocket a todos los navegadores suscritos al canal de esa subasta; **Echo** actualiza la pantalla de cada postor.
7. Al vencer el plazo, el **planificador** cierra la subasta y encola la notificación al ganador.

Aparecen aquí casi todos los conceptos del documento: concurrencia (pujas simultáneas), consistencia fuerte (la puja más alta), comunicación asíncrona (colas), tiempo real (WebSocket), escalamiento horizontal (varias instancias detrás de Nginx) y tolerancia a fallos (si Reverb se reinicia, las pujas siguen guardándose y los clientes se reconectan).

Dato curioso · Laravel y su servidor en PHP

Durante años, para tener WebSockets en Laravel había que pagar un servicio externo como Pusher o levantar un servidor en Node.js. Reverb, creado por Joe Dixon y presentado con Laravel 11, está escrito en PHP sobre ReactPHP, un bucle de eventos que permite a PHP mantener miles de conexiones abiertas. Además habla el mismo protocolo que Pusher, por lo que Echo funciona con ambos sin cambios.

11. Tendencias actuales

- **Infraestructura para inteligencia artificial:** el 66 % de las organizaciones que alojan modelos de IA generativa usa Kubernetes para gestionar la inferencia (CNCF, 2026). Entrenar y servir modelos es, en esencia, un problema de sistemas distribuidos.

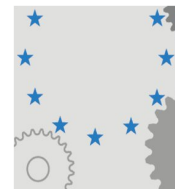
- **Exaescala:** las supercomputadoras superan el trillón de operaciones de punto flotante por segundo; la cooperación de millones de núcleos es un reto de comunicación y tolerancia a fallos (TOP500, 2026).
- **Edge computing:** mover el procesamiento cerca del usuario o del sensor para reducir la latencia.
- **Platform engineering:** equipos internos que construyen plataformas para que los desarrolladores desplieguen servicios sin dominar toda la complejidad de la infraestructura.
- **Cultura antes que tecnología:** por primera vez, la encuesta de la CNCF (2026) identifica los cambios culturales en los equipos, y no la complejidad técnica, como la principal barrera para adoptar arquitecturas nativas de la nube.

12. Glosario

Término	Definición breve
Balanceador de carga	Componente que reparte peticiones entre varias instancias de un servicio.
Broker de mensajes	Intermediario que recibe, almacena y entrega mensajes entre productores y consumidores.
CDN	Red de servidores distribuidos geográficamente que entrega contenido estático cerca del usuario.
Consenso	Acuerdo entre nodos sobre un valor, aun cuando algunos fallen.
Evento	Registro de algo que ocurrió (“se registró una puja”) al que otros componentes pueden reaccionar.
Idempotencia	Propiedad de una operación que produce el mismo resultado si se ejecuta una o varias veces.
Latencia	Tiempo que tarda un mensaje en ir de un punto a otro.
Middleware	Software intermedio que facilita la comunicación entre componentes distribuidos.
Nodo	Cada computador, máquina virtual o contenedor que participa en el sistema.
Partición de red	Situación en la que un grupo de nodos no puede comunicarse con otro.
Réplica	Copia de un dato o servicio en otro nodo.
Worker	Proceso que toma trabajos de una cola y los ejecuta.

Referencias

- Burns, B. (2025). *Designing distributed systems: Patterns and paradigms for scalable, reliable systems using Kubernetes* (2.ª ed.). O'Reilly Media.
- Cloud Native Computing Foundation. (2026, 20 de enero). *Kubernetes established as the de facto “operating system” for AI as production use hits 82% in 2025 CNCF Annual Cloud Native Survey*. <https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/>
- Gorton, I. (2022). *Foundations of scalable systems: Designing distributed architectures*. O'Reilly Media.



Joshi, U. (2023). *Patterns of distributed systems*. Addison-Wesley.

Kleppmann, M., y Riccomini, C. (2026). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems* (2.ª ed.). O'Reilly Media.

Laravel. (2026a). *Broadcasting* (documentación de Laravel 13.x).
<https://laravel.com/docs/13.x/broadcasting>

Laravel. (2026b). *Laravel Reverb* (documentación de Laravel 13.x).
<https://laravel.com/docs/13.x/reverb>

Laravel. (2026c). *Queues* (documentación de Laravel 13.x). <https://laravel.com/docs/13.x/queues>

Richards, M., y Ford, N. (2025). *Fundamentals of software architecture: A modern engineering approach* (2.ª ed.). O'Reilly Media.

TOP500. (2026, 23 de junio). *LineShine debuts at No. 1 as the TOP500 enters a new global exascale era*. <https://top500.org/news/lineshine-debuts-no-1-top500-enters-new-global-exascale-era/>

van Steen, M., y Tanenbaum, A. S. (2023). *Distributed systems* (4.ª ed.). distributed-systems.net.
<https://www.distributed-systems.net>

Vitillo, R. (2022). *Understanding distributed systems: What every developer should know about large distributed applications* (2.ª ed.). Autoedición.

`wsl --install (powershell)`

`wsl --status`

Docker desktop --- <https://www.docker.com/products/docker-desktop/> - descargar Docker AMD64

docker run hello-world

docker compose version